



Open in app

Get started



Published in Towards Data Science

You have **1** free member-only story left this month.

[Sign up for Medium and get an extra one](#)



Kenneth Leung

Follow

Dec 28, 2020 · 11 min read ★ · Listen



Save



HANDS-ON TUTORIALS

Russian Car Plate Detection with OpenCV and TesseractOCR

Detect, recognize and extract car license plate numbers with the power of computer vision (A step by step tutorial)



[Open in app](#)[Get started](#)

Table of Contents

- 1) [Motivation and Introduction](#)
- 2) [Getting Started](#)
- 3) [Car Plate Detection with OpenCV and Haar Cascade](#)
- 4) [Plate Number Recognition and Extraction with TesseractOCR](#)

Click [here](#) to view the Jupyter Notebook, and [here](#) to visit the GitHub repo.

Motivation and Introduction

When we talk about AI, computer vision is definitely one of the top applications in people's minds. Hype aside, I have always been fascinated by it because it pertains to human sight, arguably the most important organ of human sense.

I have come across plenty of computer vision projects involving the human face and/or body. As such, I decided to pursue a project on car license plates instead, which is slightly less explored.

Another reason I worked on this topic is that right after detecting the license plate, it allows for a natural transition into the use of optical character recognition (OCR) to recognize, extract, and display the detected car plate number. Let's get started!

Getting Started

We will be using Python Jupyter notebook to build our project, along with two open-source software to make the magic happen, namely OpenCV and TesseractOCR. Before we proceed further, here are the steps to get these tools fully set up on your machine.

(i) OpenCV

[OpenCV](#) (Open Source Computer Vision Library) is an open-source computer vision



[Open in app](#)[Get started](#)

We install the Python version of OpenCV (in your virtual environment) with the following command:

```
pip install opencv-python
```

(ii) Haar Cascade XML File

Besides installing the OpenCV library, another important thing to retrieve is the **Haar Cascade XML** file.

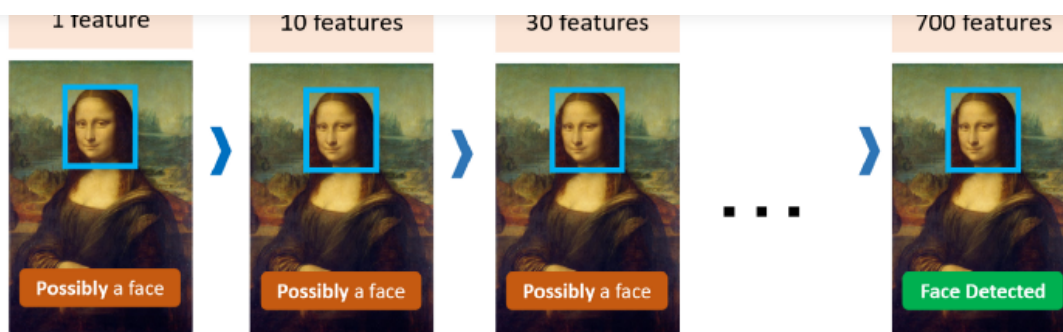
Let's first talk about the theory behind Haar Cascades since it is an important concept. In 2001, Paul Viola and Michael Jones came up with the object detection technique using Haar feature-based cascade classifiers. It is a machine learning based approach (involving AdaBoost) where a cascade function is trained from many positive and negative images. It extracts numerical values for features (e.g. edges, lines) efficiently with the concept of **integral image** (or summed-area table), which trumps the default computationally-heavy way of subtracting sums of pixels across multiple regions of an entire image.

In addition, it uses the '**Cascade of Classifiers**'. This means that instead of applying hundreds of classifiers for the many features within the image at one go (which is very inefficient), the classifiers are applied one by one.

Take for example, an image of a human face. If the first classifier for the 'eyes' feature has failed (i.e. fail to detect human eyes in the image), the algorithm does not bother applying the subsequent classifiers (e.g., nose, mouth, etc.). Instead, it stops and declares that no face is detected.

On the other hand, if this first 'eyes' feature is detected, the algorithm will apply the second stage of feature classification and continue with the classification process. In the end, if the image passes all classification stages, the algorithm can declare that a face region is present.



[Open in app](#)[Get started](#)

Visual representation of Haar Cascade process

OpenCV actually comes with pre-trained XML files of various Haar Cascades, where each XML file contains the feature set. We will be using the Haar Cascade XML file containing the features for Russian car plates, and here's how you can **download the Haar Cascade XML file**:

1. Visit the OpenCV GitHub page containing the Russian car plate Haar Cascade by clicking [here](#).
2. **Right-click** on the screen (which should be displaying a wall of text with the top line being `<?xml version="1.0"?>`), and click '**Save as..**'
3. In the Save option pop-up, you should see the default file name of '*haarcascade_russian_plate_number*' and file type '*XML document*'. Leave them as the default, and save this XML file in the path where your Jupyter notebook is located.

To explore other XML files available for experimentation, check out the OpenCV Haar Cascades GitHub resource [here](#). If you would like to find out more about the theory of Haar Cascade, visit the OpenCV tutorial [here](#).

(iii) TesseractOCR

TesseractOCR is an open-source optical character recognition (OCR) engine. It is recognized as one of the most popular and most accurate open-source OCR engines. The fun fact is that this engine actually started as proprietary software developed by Hewlett Packard but was later open-sourced in 2005, and Google has since sponsored its development.



[Open in app](#)[Get started](#)

downloaded the 27 Nov 2020 (64 bit) version (tesseract-ocr-w64-setup-v5.0.0-alpha.20201127.exe)

2. Run the downloaded installer and take note of where the application is installed. For me, I installed it inside folder `D:\Program Files\Tesseract-OCR`. We will be using this folder path later, which is important because we will need to point directly to the `tesseract.exe` inside the folder.
3. Install the Python version of TesseractOCR (i.e. PyTesseract) in your environment with the following command:

```
pip install pytesseract
```

We can then initialize the above dependencies with the following code:

```
# Import dependencies
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

import cv2 # This is the OpenCV Python library
import pytesseract # This is the TesseractOCR Python library

# Set Tesseract CMD path to the location of tesseract.exe file
pytesseract.pytesseract.tesseract_cmd = r'C:\Program
Files\Tesseract-OCR\tesseract.exe'
```

Car Plate Detection with OpenCV and Haar Cascade

First of all, we import the input car image we want to work with. Because OpenCV imports images as BGR (Blue-Green-Red) format by default, we will need to run `cv2.cvtColor` to switch it to RGB format before we ask *matplotlib* to display the image.



[Open in app](#)[Get started](#)

Our input car image. Source: [“FAB Design!”](#) by [Niklas Emmerich](#) licensed under [CC BY-NC-SA 2.0](#)

It's now time to bring in the Haar Cascade feature set (XML file) for Russian car plates, using OpenCV's CascadeClassifier function.

```
# Import Haar Cascade XML file for Russian car plate numbers
carplate_haar_cascade = cv2.CascadeClassifier('./haar_cascades/haarcascade_russian_plate_number.xml')
```

Next, we make use of the `detectMultiScale` method of the CascadeClassifier to run the detection.

```
# Setup function to detect car plate
def carplate_detect(image):
```



[Open in app](#)[Get started](#)

```
r=1.1, minNeighbors=3)

for x,y,w,h in carplate_rects:
    cv2.rectangle(carplate_overlay, (x,y), (x+w,y+h), (255,0,0),
5)

    return carplate_overlay
```

Let's briefly talk about the OpenCV `detectMultiScale` method. The method allows us to detect objects of different sizes in the input image, and it returns a list of **rectangle bounds** where objects are detected. For each rectangle, there will be 4 values returned, and they correspond to the following respectively:

- x-coordinate of the bottom-left corner of rectangle (x)
- y-coordinate of the bottom-left corner of rectangle (y)
- width of rectangle (w)
- height of the rectangle (h)

The key parameters involved in the `detectMultiScale` function are **scaleFactor** and **minNeighbors**.

- **scaleFactor** specifies how much the image size is reduced at each image scale (as part of the scale pyramid, which is a multi-scale representation of an image). In essence, when object detection models are trained, they are trained to detect objects (i.e. car plates in our case) of a fixed size and might miss car plates that are bigger or smaller than expected. As part of the scale pyramid, the image is resized several times in the hopes that a car plate will end up being a “detectable” size. I used the default scale factor of **1.1**, which means that OpenCV will scale the image down by 10% to try and match the car plates better.
- **minNeighbors** allows us to specify how many neighbors each candidate rectangle should have for the candidate rectangle to be retained. In simpler terms, this parameter influences the quality of the detected objects. A higher



[Open in app](#)[Get started](#)

We can tune these two parameters accordingly to improve your results. For more details, check out the OpenCV documentation [here](#).

Let's now run our function for car plate detection:

```
detected_carplate_img = carplate_detect(carplate_img_rgb)
plt.imshow(detected_carplate_img)
```



Input image with red rectangle bounds around the detected car license plate

We can see that our function worked a treat! The car license plate has been successfully detected and bounded by a red rectangle. Our next step is to focus on the license plate and work towards extracting the numbers and text of the car plate using OCR capabilities.



[Open in app](#)[Get started](#)

(I) Extract car license plate and enlarge image

To ensure the success of the OCR function, we need to perform a series of **image processing steps**. Let's start by isolating the car license plate as an image. We do that by setting up a function similar to what we have done earlier with car plate detection, just that this time we will be extracting and returning the region of interest (car plate) as a new image.

```
# Create function to retrieve only the car plate region itself
def carplate_extract(image):

    carplate_rects =
    carplate_haar_cascade.detectMultiScale(image, scaleFactor=1.1,
    minNeighbors=5)

    for x,y,w,h in carplate_rects:
        carplate_img = image[y+15:y+h-10 ,x+15:x+w-20] # Adjusted
        to extract specific region of interest i.e. car license plate

    return carplate_img
```

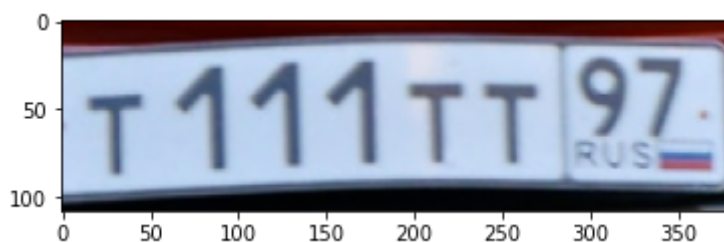
In addition, we will also want to enlarge our car license plate (using OpenCV's `resize` method) since it was only a small segment of the original input image.

```
# Enlarge image for further processing later on
def enlarge_img(image, scale_percent):
    width = int(image.shape[1] * scale_percent / 100)
    height = int(image.shape[0] * scale_percent / 100)
    dim = (width, height)
    resized_image = cv2.resize(image, dim, interpolation =
    cv2.INTER_AREA)

    return resized_image
```

Running these two functions will give us our region of interest, the car license plate itself (termed as `carplate_extract_img`).



[Open in app](#)[Get started](#)

Extracted region of interest — Car license plate

(ii) Convert to gray-scale

Next up is to convert the image from RGB color to grayscale. The purpose of this is to decrease the amount of colors in the image, which may interfere with the OCR detection. This is because we want to focus on the important edges and shapes of the image, and converting to grayscale helps to optimize this detection process.

```
# Convert image to grayscale
carplate_extract_img_gray = cv2.cvtColor(carplate_extract_img,
cv2.COLOR_RGB2GRAY)
plt.axis('off')
plt.imshow(carplate_extract_img_gray, cmap = 'gray');
```



Grayscale image of car license plate

(iii) Smoothing (aka blurring or de-noising)

Next, we apply smoothing on the image. The smoothing technique helps to get rid of noise and makes the application focus on the general details of the image. This is essentially what we call a **denoising** step, making the text characters in the image more distinct and recognizable.



[Open in app](#)[Get started](#)

demonstrated here.

```
# Apply median blur
carplate_extract_img_gray_blur =
cv2.medianBlur(carplate_extract_img_gray,3) # kernel size 3
plt.axis('off')
plt.imshow(carplate_extract_img_gray_blur, cmap = 'gray');
```



Smoothed grayscale image of car license plate

The visual difference is perhaps subtle, but if you were to compare the two images closely, you would notice that the edges of the characters are slightly smoother and less jagged. After these transformations, our image is now ready for OCR application. For other images, if a higher degree of smoothing is required, you can always increase the kernel size from the size 3 I used here. Let's pass the image into the PyTesseract `image_to_string` function:

```
# Display the text extracted from the car plate
print(pyesseract.image_to_string(carplate_extract_img_gray_blur,
                                config = f'--psm 8 --oem 3 -c
                                tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'))
```

T111TT97

Printed output of `pyesseract.image_to_string` function

Woohoo, the printed text of **'T111TT97'** does match the characters on our car



[Open in app](#)[Get started](#)

(aka **flags**):

- `tessedit_char_whitelist` helps to restrict the TesseractOCR function to a set of pre-defined (white-listed) characters. Since we know that the license plate text ranges from 0–9 and A-Z, we can specify it.
- `--oem` refers to **OCR Engine Mode (OEM)**, and the value corresponds to whichever engine mode we want. For this case, I used the default engine based on what is available, which is mode number **3** (`--oem 3`). Here are the details of the various OCR Engine Modes we can utilize:

`--oem N`

Specify OCR Engine mode. The options for *N* are:

```

0 = Original Tesseract only.
1 = Neural nets LSTM only.
2 = Tesseract + LSTM.
3 = Default, based on what is available.

```

Documentation for OEM options. Source: [Tesseract Github](#)

- `--psm` refers to **Page Segmentation Modes (PSM)**, which allows us to get TesseractOCR to run a subset of layout analysis and assume a certain form of an image. Details of the various supported options are shown below:

`--psm N`

Set Tesseract to only run a subset of layout analysis and assume a certain form of image. The options for *N* are:

```

0 = Orientation and script detection (OSD) only.
1 = Automatic page segmentation with OSD.
2 = Automatic page segmentation, but no OSD, or OCR. (not implemented)
3 = Fully automatic page segmentation, but no OSD. (Default)
4 = Assume a single column of text of variable sizes.
5 = Assume a single uniform block of vertically aligned text.
6 = Assume a single uniform block of text.
7 = Treat the image as a single text line.
8 = Treat the image as a single word.
9 = Treat the image as a single word in a circle.
10 = Treat the image as a single character.
11 = Sparse text. Find as much text as possible in no particular order.
12 = Sparse text with OSD.
13 = Raw line. Treat the image as a single text line

```



[Open in app](#)[Get started](#)

Since our car license image is to be treated as a **single word** (i.e. PSM option number **8**), I used the value of `--psm 8` within the `config` parameter. Out of curiosity, I also experimented with all the different possible PSM options:

```
# Testing all PSM values
for i in range(3,14):
    print(f'PSM: {i}')

print(pyesseract.image_to_string(carplate_extract_img_gray_blur,
                                config = f'--psm {i} --oem 3
-c tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'))
```

PSM: 3 T111TTI2	PSM: 9 RRRRESS
PSM: 4 T111TTI2	PSM: 10 T111TTI2
PSM: 5 0	PSM: 11 T111TT92
PSM: 6 T111TTI2	PSM: 12 T111TT92
PSM: 7 T111TTI2	PSM: 13 T111TT97
PSM: 8 T111TT97	

Output strings based on different PSM options

We can see that different PSM options lead to varying outcomes, so it is crucial to set



[Open in app](#)[Get started](#)

PyTesseract) within Python and how to use their powerful in-built functions to detect car license plates (of Russian cars) and extract the text from these number plates.

On top of that, we also discussed several theoretical concepts such as Haar Cascades, multi-scale detection parameters, image processing for optimized recognition, and TesseractOCR's page segmentation modes (PSM).

This project serves as a stepping stone for larger scale (and more advanced) computer vision projects, such as bulk extraction of car license plate text from large image quantities and applying these concepts on video files or live feed.

Before you go

I welcome you to **join me on a data science learning journey!** Follow this [Medium](#) page and check out my [GitHub](#) to stay in the loop of more exciting data science content. Have fun applying OCR on vehicle plate numbers!

Evaluate OCR Output Quality with Character Error Rate (CER) and Word Error Rate (WER)

Key concepts, examples and Python implementation of measuring Optical Character Recognition output quality

towardsdatascience.com

Top Python libraries for Image Augmentation in Computer Vision

Featuring the best augmentation libraries (along with sample codes) for your next computer vision project

towardsdatascience.com





Open in app

Get started

Introduction

A step-by-step guide on using PyTorch Ignite to simplify your PyTorch deep learning implementation

towardsdatascience.com



127



References

- <https://nanonets.com/blog/ocr-with-tesseract>
- <https://stackoverflow.com/questions/44619077/pytesseract-ocr-multiple-config-options>

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. [Take a look.](#)



Get this newsletter

